Homework 08

Eric Chen

All code is in Python. No classifier is fit in Questions 1 or 2; every quantity comes from the given linear scores.

Question 1: Boundary and margin

Use `margin-geometry-20.csv` and the score $f(x) = 0.8x_1 + 0.6x_2$. (a) Report each observation's distance to the boundary, its functional margin $yf(x)$, and whether it is inside, on, or beyond the margin for its class. (b) Plot the observations, the boundary, and the margin lines, and identify the observations inside the margin. (c) Count the observations in each group and identify the misclassified observations.

Solution

For a linear score $f(x) = \beta_0 + x^T\beta$, the signed distance from x to the boundary $f(x) = 0$ is $f(x)/\|\beta\|_2$. Here $\beta_0 = 0$ and $\|\beta\|_2 = \sqrt{0.8^2 + 0.6^2} = 1$, so the distance is $|f(x)|$, and $yf(x)$ is the signed distance on the side of the observation's own class. An observation is **beyond** the margin if $yf(x) > 1$, **on** it if $yf(x) = 1$, and **inside** it if $yf(x) < 1$. Observations with $yf(x) < 0$ are inside the margin and also on the wrong side of the boundary, so they are misclassified.

```
import warnings; warnings.filterwarnings("ignore")
import numpy as np, pandas as pd, matplotlib.pyplot as plt

d = pd.read_csv("data/margin-geometry-20.csv")
beta = np.array([0.8, 0.6])
d["f"] = d[["x1", "x2"]].to_numpy() @ beta
d["distance"] = np.abs(d["f"]) / np.linalg.norm(beta)
d["yf"] = d["y"] * d["f"]

# Round before comparing with 1 so that values such as 0.544 + 0.456 count as exactly 1
yf = d["yf"].round(10)
d["position"] = np.select([yf < 1, yf == 1], ["inside", "on"], "beyond")
d["classified"] = np.where(yf > 0, "correct", "wrong")
```

(a) The table lists all 20 observations in the order given in the data.

Table 1: Distance to the boundary, functional margin, and margin position for each observation.

```
tab = d[["observation", "x1", "x2", "y", "f", "distance", "yf", "position", "classified"]]
tab = tab.rename(columns={"observation": "obs", "f": "f(x)", "yf": "y f(x)"})
print(tab.to_string(index=False, float_format=lambda v: f"{v:.2f}"))
```

```
 obs    x1    x2  y  f(x)  distance  y f(x) position classified
   1  0.68  0.76  1  1.00      1.00    1.00       on    correct
   2  1.80 -0.15  1  1.35      1.35    1.35   beyond    correct
   3  1.38  0.66  1  1.50      1.50    1.50   beyond    correct
   4  0.88  1.66  1  1.70      1.70    1.70   beyond    correct
   5  0.62  2.34  1  1.90      1.90    1.90   beyond    correct
   6  2.78 -0.04  1  2.20      2.20    2.20   beyond    correct
   7  1.86  1.52  1  2.40      2.40    2.40   beyond    correct
   8  0.50  0.40  1  0.64      0.64    0.64   inside    correct
   9  0.10  0.20  1  0.20      0.20    0.20   inside    correct
  10 -0.40  0.20  1 -0.20      0.20   -0.20   inside      wrong
  11 -0.92 -0.44 -1 -1.00      1.00    1.00       on    correct
  12 -0.50 -1.50 -1 -1.30      1.30    1.30   beyond    correct
  13 -1.90 -0.05 -1 -1.55      1.55    1.55   beyond    correct
  14 -0.44 -2.33 -1 -1.75      1.75    1.75   beyond    correct
  15 -1.98 -0.61 -1 -1.95      1.95    1.95   beyond    correct
  16 -2.74  0.07 -1 -2.15      2.15    2.15   beyond    correct
  17 -1.90 -1.55 -1 -2.45      2.45    2.45   beyond    correct
  18 -0.50 -0.40 -1 -0.64      0.64    0.64   inside    correct
  19  0.00 -0.40 -1 -0.24      0.24    0.24   inside    correct
  20  0.40 -0.20 -1  0.20      0.20   -0.20   inside      wrong
```

(b)

```
fig, ax = plt.subplots(figsize=(5.6, 5.4))
x1 = np.array([-3.2, 3.2])
# f(x) = 0.8 x1 + 0.6 x2 = c  ->  x2 = (c - 0.8 x1) / 0.6
for c, ls, lab in [(0, "-", "f(x) = 0"), (1, "--", "f(x) = ±1"), (-1, "--", None)]:
    ax.plot(x1, (c - 0.8 * x1) / 0.6, color="0.3", ls=ls, lw=1.2, label=lab)

for cls, col, mk, lab in [(1, "#0072B2", "o", "y = +1"), (-1, "#D55E00", "s", "y = −1")]:
    s = d[d["y"] == cls]
    ax.scatter(s["x1"], s["x2"], c=col, marker=mk, s=42, label=lab, zorder=3)

inside = d[d["position"] == "inside"]
ax.scatter(inside["x1"], inside["x2"], s=190, facecolors="none", edgecolors="black",
           lw=1.2, label="inside margin", zorder=4)
for _, r in d.iterrows():
    ax.annotate(int(r["observation"]), (r["x1"], r["x2"]), xytext=(4, 4),
                textcoords="offset points", fontsize=7.5)

ax.set(xlim=(-3.2, 3.2), ylim=(-3, 3), xlabel="$x_1$", ylabel="$x_2$")
ax.set_aspect("equal")
ax.legend(loc="lower right", fontsize=8, frameon=True)
fig.tight_layout(); plt.show()
```

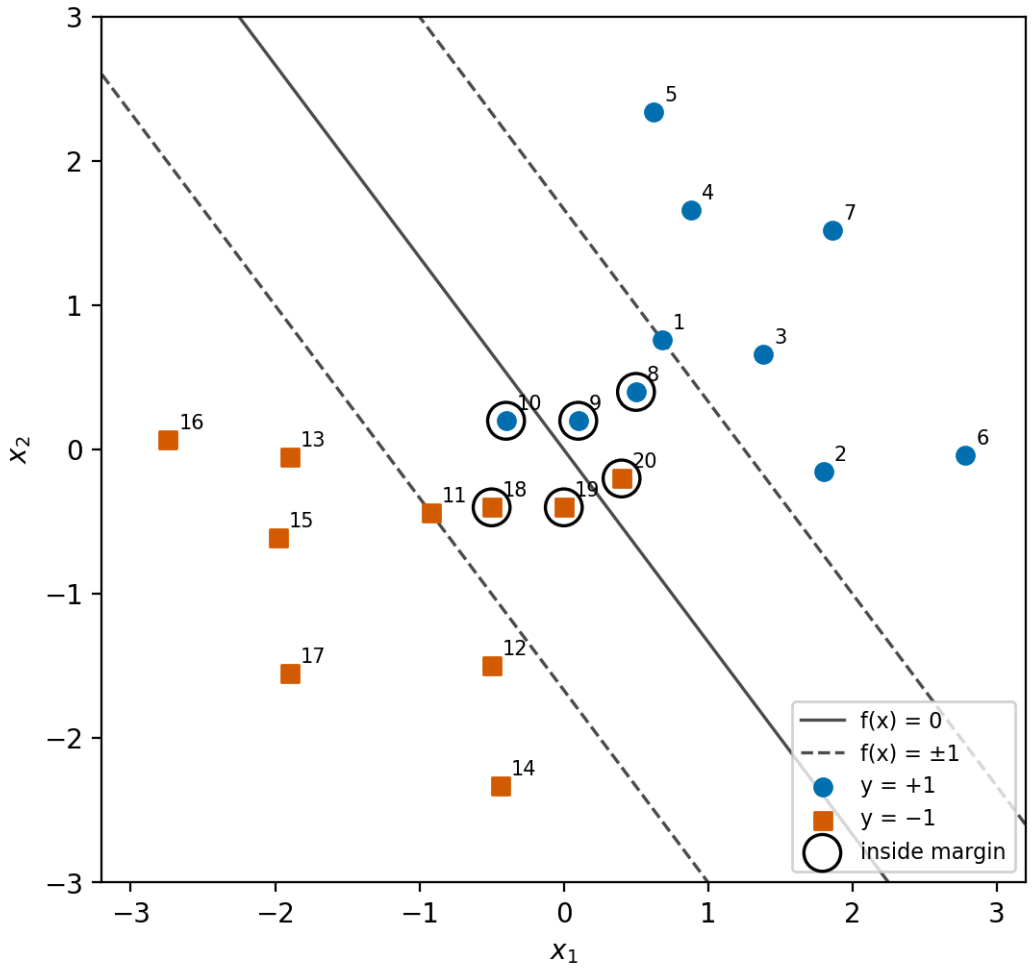


Figure 1: Observations, decision boundary f(x)=0 (solid), and margin lines f(x)=±1 (dashed). Open black circles mark the observations inside the margin.

(c)

```
print(d["position"].value_counts().reindex(["inside", "on", "beyond"]).to_string())
print("inside:", d.loc[d["position"] == "inside", "observation"].tolist())
print("on:", d.loc[d["position"] == "on", "observation"].tolist())
print("misclassified:", d.loc[d["classified"] == "wrong", "observation"].tolist())
```

```
position
inside     6
on         2
beyond    12
inside: [8, 9, 10, 18, 19, 20]
on: [1, 11]
misclassified: [10, 20]
```

- **Inside the margin: 6 observations** (8, 9, 10, 18, 19, 20), with $yf(x) < 1$.
- **On the margin: 2 observations** (1 and 11), with $yf(x) = 1$ exactly. Observation 1 lies on $f(x) = 1$ and observation 11 lies on $f(x) = -1$.
- **Beyond the margin: 12 observations** (2–7 and 12–17), with $yf(x) > 1$.

Misclassified: observations 10 and 20. Both have $yf(x) = -0.20 < 0$. Observation 10 has $y = +1$ but $f(x) = -0.20$, and observation 20 has $y = -1$ but $f(x) = 0.20$. Each lies a distance 0.20 from the boundary on the wrong side. The other four observations inside the margin (8, 9, 18, 19) are correctly classified because $0 < yf(x) < 1$. They are on the correct side of the boundary but closer than distance 1.

Question 2: Margin in four dimensions

Use `margin-geometry-4d.csv` and $f(\mathbf{x}) = \beta_0 + \mathbf{x}^T\boldsymbol{\beta}$ with $\beta_0 = -1$ and $\boldsymbol{\beta} = (1, 2, -2, 4)^T$. (a) Project the observations onto the direction of $\boldsymbol{\beta}$, plot them on one axis, mark the boundary, and find the geometric margin M. (b) Rescale the boundary so that the smallest functional margin is 1.

Solution

Let $\mathbf{u} = \boldsymbol{\beta}/\|\boldsymbol{\beta}\|_2$, where $\|\boldsymbol{\beta}\|_2 = \sqrt{1 + 4 + 4 + 16} = 5$. The projection of $\mathbf{x}$ onto this direction is the scalar $z = \mathbf{x}^T\mathbf{u}$. Because $f(\mathbf{x}) = \beta_0 + \|\boldsymbol{\beta}\|_2\, z$, the boundary $f(\mathbf{x}) = 0$ is the single point

$$z_0 = -\beta_0/\|\boldsymbol{\beta}\|_2 = 1/5 = 0.2$$

on this axis, and the signed distance from $\mathbf{x}$ to the boundary is $z - z_0 = f(\mathbf{x})/\|\boldsymbol{\beta}\|_2$. Distances in the four-dimensional space perpendicular to the boundary are therefore preserved exactly on the projected axis.

```
X = pd.read_csv("data/margin-geometry-4d.csv")
b0, b = -1.0, np.array([1, 2, -2, 4])
nb = np.linalg.norm(b)
X["z"] = X[["x1", "x2", "x3", "x4"]].to_numpy() @ b / nb
X["f"] = b0 + nb * X["z"]
X["yf"] = X["y"] * X["f"]
X["dist"] = X["yf"] / nb          # signed distance on the correct side
z0 = -b0 / nb
print(f"||beta|| = {nb:.0f}, boundary at z0 = {z0:.2f}")
print("misclassified:", int((X["yf"] <= 0).sum()))
print(X.sort_values("dist")[["observation", "y", "z", "f", "yf", "dist"]]
      .head(6).to_string(index=False, float_format=lambda v: f"{v:.2f}"))
```

```
||beta|| = 5, boundary at z0 = 0.20
misclassified: 0
 observation  y     z     f   yf  dist
           4  1  1.20  5.00 5.00  1.00
          15 -1 -0.80 -5.00 5.00  1.00
          33  1  1.20  5.00 5.00  1.00
          36 -1 -0.80 -5.00 5.00  1.00
          17 -1 -1.00 -6.00 6.00  1.20
          31  1  1.40  6.00 6.00  1.20
```

(a)

```
fig, ax = plt.subplots(figsize=(7, 2.6))
rows = {1: 0.25, -1: -0.25}
for cls, col, mk, lab in [(1, "#0072B2", "o", "y = +1"), (-1, "#D55E00", "s", "y = −1")]:
    s = X[X["y"] == cls]
    ax.scatter(s["z"], np.full(len(s), rows[cls]), c=col, marker=mk, s=34,
               alpha=0.75, label=lab, zorder=3)
M = X["dist"].min()
closest = X[np.isclose(X["dist"], M)]
ax.scatter(closest["z"], closest["y"].map(rows), s=160, facecolors="none",
           edgecolors="black", lw=1.2, label="closest (distance M)", zorder=4)
ax.axvline(z0, color="0.2", lw=1.4)
```

```
for s_ in (-1, 1):
    ax.axvline(z0 + s_ * M, color="0.4", ls="--", lw=1)
ax.annotate("boundary $z_0=0.2$", (z0, 0.55), ha="center", fontsize=8)
ax.set(ylim=(-0.6, 0.7), yticks=[], xlabel=r"projection $z=\mathbf{x}
^{\mathsf{T}}\boldsymbol{\beta}/\|\boldsymbol{\beta}\|_2$")
ax.legend(loc="lower center", bbox_to_anchor=(0.5, 1.0), ncol=3, fontsize=8, frameon=False)
fig.tight_layout(); plt.show()
```

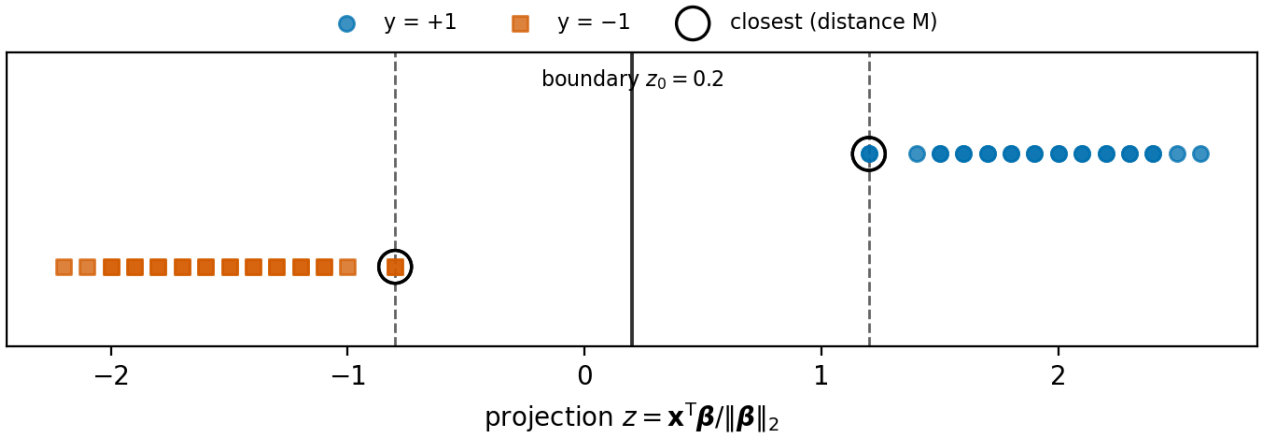


Figure 2: Projections z = xᵀβ/‖β‖ of the 50 observations, with the two classes on separate rows. The solid line is the decision boundary z₀ = 0.2. The dashed lines are a distance M = 1 away, and the open circles mark the four closest observations.

All $y = +1$ observations project to the right of $z_0 = 0.2$ and all $y = -1$ observations project to the left, so f separates the two classes with no errors. The closest observations on each side are at $z = 1.2$ (observations 4 and 33, $y = +1$) and $z = -0.8$ (observations 15 and 36, $y = -1$). Each is a distance 1.0 from z_0. The geometric margin is therefore

$$M = \min_i \frac{y_i f(\mathbf{x}_i)}{\|\boldsymbol{\beta}\|_2} = \frac{5}{5} = 1.$$

The smallest functional margin is $\min_i y_i f(\mathbf{x}_i) = 5$, attained by these four observations. The next closest observation is at distance 1.2.

(b) Multiplying $(\beta_0, \boldsymbol{\beta})$ by any $c > 0$ leaves the boundary $\{\mathbf{x} : f(\mathbf{x}) = 0\}$ and every geometric distance unchanged but multiplies each functional margin by c. To set the smallest functional margin to 1, take $c = 1/\min_i y_i f(\mathbf{x}_i) = 1/5$:

$$\boldsymbol{\beta}^* = \frac{1}{5}(1, 2, -2, 4)^T = (0.2, 0.4, -0.4, 0.8)^T, \qquad \beta_0^* = -\frac{1}{5} = -0.2.$$

```
c = 1 / X["yf"].min()
b_star, b0_star = c * b, c * b0
yf_star = X["y"] * (b0_star + X[["x1", "x2", "x3", "x4"]].to_numpy() @ b_star)
print("beta* =", b_star.round(2), " beta0* =", round(b0_star, 2))
print(f"min y f*(x) = {yf_star.min():.2f},  ||beta*|| = {np.linalg.norm(b_star):.2f},"
      f"  1/||beta*|| = {1 / np.linalg.norm(b_star):.2f}")
```

```
beta* = [ 0.2  0.4 -0.4  0.8]  beta0* = -0.2
min y f*(x) = 1.00,  ||beta*|| = 1.00,  1/||beta*|| = 1.00
```

Under this normalization the four closest observations (4, 15, 33, 36) lie exactly on $f^*(\mathbf{x}) = \pm 1$, and $\|\boldsymbol{\beta}^*\|_2 = 1$. This agrees with the SVM identity $M = 1/\|\boldsymbol{\beta}^*\|_2 = 1$ and with part (a).

Question 3: Peer review of a Homework 7 skill

Assigned repository: Urva-Vashi, `homework/week-07/homework-07.html`. The skill was copied without changes into a local `SKILL.md` (3,195 bytes) and applied to the SAheart data, with `chd` as the response and the other nine variables as numerical predictors (`famhist` coded 1 = Absent, 2 = Present). The run was not stopped early.

Overall assessment

The model was successfully able to create a training/test set, perform cross validation on all of the models, and determine that performing PCA was better than LDA/QDA.

Major item 1

I wish that the report had some sample of code, or at least some way to source the claims that it makes in the report

Major item 2

I also wish that the agent worked more efficiently. The agent took 3 minutes and 66.1k models on Opus 5.5, which seems pretty excessive for a relatively simple set of calculations.

Major item 3

Finally, I wish that the model's report was formatted using more "bullet-point" style writing, as this academic-style writing is a little difficult to parse at first glance.

GitHub Issue

Issue posted to the assigned repository: https://github.com/Urva-Vashi/stat432-fall2026/issues/4